

Горутины

– это легковесные потоки

Go vs OS-thread

Характеристика	Goroutine	OS Thread
Размер стека	2KB (динамически растёт)	1MB (фиксированный)
Переключение	Планировщик Go (user-space)	Ядро ОС (kernel-space)
Создание	Дешевое (тысячи)	Дорогое (сотни)
Идентификатор	Не видимы для ОС	Видимы для ОС (TID)

main() – это тоже горутина

Синтаксис

```
go func() {  
    fmt.Println("just a goroutine")  
}()
```

Про ожидание множества горутин и синхронизацию поговорим в другом блоке

Каналы

– это типизированные очереди для обмена данными между горутинами

```
ch <- v    // Send v to channels ch  
v := <-ch  // Receive from ch, and  
           // assign value to v
```

Блокировка

Блокировка – это состояние, в котором горутина не может продолжать выполнение, потому что:

- Она пытается прочитать из канала, но данных нет
- Она пытается отправить в канал, но никто не читает (или буфер полон)

В этот момент горутина ожидает, пока другая горутина не сделает необходимое действие

Что такое ожидание/блокировка на уровне рантайма?

Состояние Что значит

_Grunning	Горутина выполняется на CPU прямо сейчас
_Grunnable	Горутина готова к выполнению, ждет очереди
_Gwaiting	Горутина заблокирована (ждет канал, мьютекс, syscall)
_Gdead	Горутина завершена

Блокировка – это переход из _Grunning в _Gwaiting

Небуферизированный канал

ПРАВИЛО:

Отправка и получение должны произойти одновременно. Если одна сторона готова, а другая нет – готовая сторона блокируется

- Отправка `ch <- 42` **БЛОКИРУЕТСЯ**, пока кто-то не прочитает `<-ch`
- Чтение `val := <-ch` **БЛОКИРУЕТСЯ**, пока кто-то не отправит

Подробнее:

```
package main

import "fmt"

func sum(s []int, c chan int) {
    sum := 0
    for _, v := range s {
        sum += v
    }
    c <- sum // send sum to c
}

func main() {
    s := []int{7, 2, 8, -9, 4, 0}

    c := make(chan int)
    go sum(s[:len(s)/2], c)
    go sum(s[len(s)/2:], c)
    x, y := <-c, <-c // receive from c

    fmt.Println(x, y, x+y)
}
```

- `main()` блокируется дважды: первый раз, когда читает `x`, и второй раз, когда читает `y`.
- Отправки в горуты НЕ БЛОКИРУЮТСЯ, потому что `main()` уже ждёт.
- Порядок получения соответствует порядку отправки (кто первый отправил — того и получили).
- Канал небуферизированный, поэтому отправка и получение всегда синхронны.

Буферизированный канал:

- Отправка блокируется только когда буфер заполнен
- Чтение блокируется только когда буфер пуст

```
package main

import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 1
    ch <- 2
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Вывод:

```
1
2
```

Но если сделать так

```
package main

import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 1
    ch <- 2
    ch <- 3
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Вывод:

```
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]:
main.main()
    /tmp/sandbox2992145173/prog.go:9 +0x58
```

Или так

```
package main

import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 3
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Вывод:

3

`fatal` error: all goroutines are asleep - deadlock!

`goroutine 1 [chan receive]:`

`main.main()`

`/tmp/sandbox2773191034/prog.go:9 +0xa7`

Структура канала

```
type hchan struct {
    qcount    uint           // количество элементов в буфере
    dataqsiz  uint           // размер буфера
    buf       unsafe.Pointer // указатель на буфер
    elemsize  uint16        // размер элемента
    closed    uint32         // закрыт ли канал
    sendx     uint           // индекс отправки в буфере
    recvx     uint           // индекс получения в буфере
    recvq     waitq         // очередь получателей (sudog)
    sendq     waitq         // очередь отправителей (sudog)
    lock      mutex        // мьютекс для защиты канала
}

// Из исходников Go (runtime/chan.go)
type waitq struct {
    first *sudog
    last  *sudog
}

// Упрощенная версия из runtime
type sudog struct {
    g          *g           // указатель на горутину
    elem       unsafe.Pointer // указатель на данные (в стеке)
    next       *sudog       // следующий элемент в очереди
    prev       *sudog       // предыдущий элемент в очереди
    c          *hchan       // канал, к которому относится
    isSelect   bool         // true если используется в select
    // ...остальные поля
}
```

Если канал небуферизированный, то:

- `qcount = 0`
- `dataqsiz = 0`
- `buf = nil`
- `sendx = 0, recvx = 0`

select

select позволяет горутине ждать несколько каналов. Он работает как switch, но для каналов

Блокирующий select

(ждет, пока один из каналов не будет готов)

```
select {
case msg1 := <-ch1:
    fmt.Println(msg1)
case msg2 := <-ch2:
    fmt.Println(msg2)
}
```

Если готовы оба канала – select выберет псевдослучайный и выполнит его. Это защита от голодания.

На случай закрытия канала:

```
for {
    select {
        // лучше использовать такой подход
        // иначе не отличить закрыт канал и вернул zero-value
        // или открыт и вернул zero-value
        case msg, ok := <-ch:
            if !ok {
                fmt.Println("Канал закрыт, выхожу")
                return
            }
            fmt.Println(msg)
    }
}
```

Неблокирующий select (с default)

```
select {
case msg := <-ch:
    fmt.Println(msg)
default:
    fmt.Println("Ни один из каналов не готов, иду дальше")
}
```

select с таймаутом

time.After(duration) создает канал, в который через указанное время придет значение (текущее время)

```
// Реализация внутри пакета time (упрощенно)
func After(d Duration) <-chan Time {
    return NewTimer(d).C // создает таймер и возвращает его канал
}
```

Она возвращает канал только для чтения (<-chan Time), который:

1. Блокирует чтение, пока не пройдет указанное время
2. После истечения времени отправляет в канал текущее время
3. Канал создается один раз, и после отправки значения больше ничего не происходит

```
select {
case val := <-ch:
    fmt.Println(val)
```

```
case <-time.After(2 * time.Second):
    fmt.Println("Таймаут")
}
```

Важные моменты:

1. Утечка памяти в цикле

```
// ПЛОХО: создает новый канал на каждой итерации
for {
    select {
        case <-ch:
            // делаем что-то
        case <-time.After(1 * time.Second):
            fmt.Println("Таймаут")
    }
}
```

Проблема: на каждой итерации создается новый таймер, и пока не истечет 1 секунда, старые таймеры продолжают жить -> утечка памяти.

Решение: использовать `time.NewTimer()` и сбрасывать его

```
// ХОРОШО: переиспользуем один таймер
timer := time.NewTimer(1 * time.Second)
defer timer.Stop()

for {
    select {
        case <-ch:
            // делаем что-то
            timer.Reset(1 * time.Second) // сбрасываем таймер
        case <-timer.C:
            fmt.Println("Таймаут")
    }
}
```

2. Канал создается ДО начала чтения

```
// time.After() создает канал сразу
ch := time.After(5 * time.Second)

// Только через 5 секунд канал получит значение
// До этого - блокировка
<-ch
```

Важно: таймер запускается в момент вызова `time.After()`, а не в момент чтения из канала.

```
start := time.Now()
ch := time.After(5 * time.Second)
time.Sleep(2 * time.Second) // прошло 2 секунды
<-ch                       // подождет еще 3 секунды (всего 5)
fmt.Println(time.Since(start)) // ~5 секунд
```

3. После отправки значения канал не закрывается

```
ch := time.After(2 * time.Second)
<-ch // получили время

// Канал все еще открыт, но больше значений не будет
```

```
// Второе чтение заблокирует навсегда!  
<-ch // БЛОКИРОВКА НАВСЕГДА
```

Случай когда можно использовать тикер

```
func simpleTickerExample() {  
    // Создаем тикер с интервалом 2 секунды  
    ticker := time.NewTicker(2 * time.Second)  
    defer ticker.Stop() // ВАЖНО: всегда останавливаем тикер!  
  
    // Счетчик для выхода из цикла  
    count := 0  
  
    for {  
        select {  
        case t := <-ticker.C:  
            count++  
            fmt.Printf("Тик #%d в %s\n", count, t.Format("15:04:05"))  
  
            // Выходим после 5 тиков (для демонстрации)  
            if count >= 5 {  
                fmt.Println("Тикер остановлен")  
                return  
            }  
  
        case <-ctx.Done(): // если используете контекст  
            fmt.Println("Остановка по контексту")  
            return  
        }  
    }  
}
```

Случай когда таймер лучше тикера

```
func adaptivePolling() {  
    timer := time.NewTimer(1 * time.Second)  
    defer timer.Stop()  
  
    for {  
        select {  
        case <-timer.C:  
            data := fetchData()  
  
            // Меняем интервал в зависимости от данных  
            if data.IsEmpty() {  
                timer.Reset(5 * time.Second) // Если данных нет, проверяем реже  
            } else {  
                timer.Reset(1 * time.Second) // Если данные есть, проверяем чаще  
            }  
  
            processData(data)  
        case <-ctx.Done():  
            return  
        }  
    }  
}
```

Аналог с тикером (но между ticker.Stop() и созданием нового может быть потерян тик)

Плохая практика

```
func adaptivePollingWithTicker() {
    // Начальный интервал
    interval := 1 * time.Second
    ticker := time.NewTicker(interval)
    defer ticker.Stop()

    for {
        select {
        case <-ticker.C:
            data := fetchData()

            // Меняем интервал в зависимости от данных
            newInterval := 1 * time.Second
            if data.IsEmpty() {
                newInterval = 5 * time.Second // Если данных нет, проверяем реже
            } else {
                newInterval = 1 * time.Second // Если данные есть, проверяем чаще
            }

            // Если интервал изменился – пересоздаем тикер
            if newInterval != interval {
                interval = newInterval
                ticker.Stop() // Останавливаем старый
                ticker = time.NewTicker(interval) // Создаем новый
            }

            processData(data)

        case <-ctx.Done():
            ticker.Stop()
            return
        }
    }
}
```


close() and range

close()

```
ch := make(chan int)
close(ch)
```

Заккрытие канала – это сигнал получателям, что данные закончились

Кто закрывает?

Только отправитель. Если получатель закроет канал, то вызовет panic

Чтение из закрытого канала:

```
val, ok := <-ch
// ok == false, если канал закрыт и пуст
// val == zero-value для типа (для int это 0)
```

В общем, можно читать пока данные не закончатся, но когда они закончатся мы получим val равное zero-value и ok равное false

range

range автоматически завершается, когда канал закрыт и все данные прочитаны

```
ch := make(chan int, 3)
ch <- 1
ch <- 2
ch <- 3
// ch <- 4 вызовет deadlock и тогда
// код ниже в исполнении не выполнится
close(ch)

for val := range ch {
    // 1, 2, 3
    fmt.Println(val)
}
```

Особенности:

1. Чтение до закрытия канала

```
package main

import "fmt"

func main() {
    ch := make(chan int)

    go func() {
        for i := 0; i < 5; i++ {
            ch <- i
        }
        close(ch) // обязательно закрыть!
        fmt.Println("Канал закрыт")
    }()

    for v := range ch {
        fmt.Println(v) // 0, 1, 2, 3, 4
    }
}
```

```
    fmt.Println("Цикл завершён")
}
```

В данном примере важно прояснить вывод:

```
0
1
2
3
Канал закрыт
4
Цикл завершён
```

range блокирует выполнение main до получения данных в канал, а чтение выполняется до закрытия канала, поэтому сообщение о закрытии канала может быть как до вывода данных, так и после, но оно **ВСЕГДА** будет **ДО** сообщения о завершении цикла, тк закрытие канала, происходит до конца цикла.

2. Блокировка при отсутствии данных

```
ch := make(chan int)
// Нет горутины, которая пишет в канал

for v := range ch { // БЛОКИРУЕТСЯ НАВСЕГДА!
    fmt.Println(v)
}
// Сюда никогда не дойдём
```

Deadlock

– это ситуация, в которой программа заблокирована навсегда, потому что горютины ждут события, которое никогда не произойдет

Программа в таком случае падает с сообщением:

```
fatal error: all goroutines are asleep - deadlock!
```

Случаи дедлока

1. Отправка в небуф. канал без получателя:

```
func main() {  
    ch := make(chan int)  
    ch <- 42 // deadlock  
}
```

Причина: main() – единственная горютина, а получателя нет

2. Получение из небуферизированного канала без отправителя

```
func main() {  
    ch := make(chan int)  
    <-ch // deadlock  
}
```

Причина: main() блокируется на чтении, а отправителя нет.

3. Заполненный буферизированный канал

```
func main() {  
    ch := make(chan int, 2)  
    ch <- 1  
    ch <- 2  
    ch <- 3 // deadlock  
}
```

Причина: Буфер полон, а получателя нет

4. Чтение из пустого буферизированного канала

```
func main() {  
    ch := make(chan int, 2)  
    <-ch // deadlock  
}
```

Причина: Буфер пуст, а отправителя нет

5. Взаимная блокировка двух горютин

```
func main() {  
    ch1 := make(chan int)  
    ch2 := make(chan int)  
  
    go func() {  
        ch1 <- 42 // ждет, пока main не прочтает из ch1  
        <-ch2     // ждет, пока main не отправит в ch2  
    }()  
  
    ch2 <- 100    // ждет, пока горютина не прочтает из ch2  
    <-ch1         // ждет, пока горютина не отправит в ch1  
}
```

Причина: Горутина ждет, чтобы main принял данные из ch1, а затем ждет данные main из ch2. А main сначала отправляет данные в ch2, а затем ждет данные от горутины из ch1. При таком порядке мы получаем взаимную блокировку навсегда

Вывод:

Проблема	Решение
Нет получателя для отправки	Нужна горутина получатель
Нет отправителя для получения	Нужна горутина отправитель
Буфер переполнен	Не переполнять буфер, увеличить буфер, запустить получателя
Взаимная блокировка	Использовать select с таймаутом или default

nil-каналы

```
var ch chan int // nil-channel
ch <- 42        // deadlock
<-ch           // deadlock
close(ch)       // panic
```

Зачем же их использовать?

1. Отключение веток в select

```
func main() {
    ch1 := make(chan int)
    ch2 := make(chan int)

    // ... какая-то логика, которая решает, что ch2 больше не нужен
    ch2 = nil // <-- ОТКЛЮЧАЕМ КАНАЛ

    for {
        select {
            case v, ok := <-ch1:
                if !ok {
                    ch1 = nil // отключаем закрытый канал
                    continue
                }
                fmt.Println("ch1:", v)
            case v, ok := <-ch2:
                // ЭТА ВЕТКА БОЛЬШЕ НИКОГДА НЕ ВЫПОЛНИТСЯ!
                // Потому что чтение из nil-канала блокирует её навечно.
                fmt.Println("ch2:", v)
        }
    }
}
```

2. Ленивая инициализация (Singleton для каналов)

```
type Worker struct {
    ch chan Task
}

func (w *Worker) Start() {
    if w.ch == nil {
        w.ch = make(chan Task, 10) // ленивое создание
```

```

    }
    // ... работаем
}

```

3. Имитация глушилки в тестах (тестируем как ведет себя код, когда данные не приходят или для блокировки ветки select)
4. Безопасная остановка горутин (костыль вместо context)

```

func worker(stop <-chan struct{}) {
    for {
        select {
        case <-stop:
            return
        default:
            // работаем
        }
    }
}
// Если передать nil, горутина никогда не завершится по этому каналу
// (иногда это нужно для тестов или отладки)

```

Направленные каналы

```

// Только для отправки
func producer(ch chan<- int) {
    ch <- 42
    // <-ch // ОШИБКА: нельзя читать из send-only
}

// Только для чтения
func consumer(ch <-chan int) {
    val := <-ch
    // ch <- 42 // ОШИБКА: нельзя писать в receive-only
}

```

Это контракт на уровне типов. Компилятор не даст случайно прочитать там, где нужно только писать, и наоборот.

Про FAN-IN и FAN-OUT читай в Concurrency